

基于 YOLO11 识别红绿灯

二. 模型训练

3、训练模型

1) 环境准备

下载 ultralytics/ultralytics 仓库，并安装相应环境和依赖。

(如计算机没有 git 程序，先从 <https://git-scm.com/downloads/win> 下载 git 安装程序，并安装。)

```
git clone https://github.com/ultralytics/ultralytics.git
```

```
(base) C:\Users\hchuang>git clone https://github.com/ultralytics/ultralytics.git
Cloning into 'ultralytics'...
remote: Enumerating objects: 76315, done.
remote: Counting objects: 100% (225/225), done.
remote: Compressing objects: 100% (150/150), done.
remote: Total 76315 (delta 138), reused 84 (delta 75), pack-reused 76090 (from 2)
Receiving objects: 100% (76315/76315), 41.22 MiB | 7.12 MiB/s, done.
Resolving deltas: 100% (57330/57330), done.
```

进入本地仓库，下载官方的预训练权重（260 万参数的 YOLO11n-Detect 模型）

```
cd ultralytics
```

```
wget https://github.com/ultralytics/assets/releases/download/v8.3.0/yolo11n.pt
```

(注意，计算机上没有安装 wget 程序，直接拷贝 yolo11n.pt 到 ultralytics 目录下)

```
(base) C:\Users\hchuang\ultralytics>wget https://bgithub.xyz/ultralytics/assets/releases/download/v8.3.0/yolo11n.pt
--2025-04-13 10:46:26-- https://bgithub.xyz/ultralytics/assets/releases/download/v8.3.0/yolo11n.pt
Resolving bgithub.xyz (bgithub.xyz)... 51.158.204.132
Connecting to bgithub.xyz (bgithub.xyz)|51.158.204.132|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://objects.bgithub.xyz/github-production-release-asset-2e65be/521807533/34b70ade-b6eb-4179-a60f-d6494307226b?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=releaseassetproduction%2F20250413%2Fus-east-1%2Fs3%2Faws4_request&X-Amz-Date=20250413T024624Z&X-Amz-Expires=300&X-Amz-Signature=6ff6748bcd6a9ed033cd7a55a36d384b8d15a07da6ebb894f1b924da4b9aa56b5X-Amz-SignedHeaders=host&response-content-disposition=attachment%3B%20filename%3Dyolo11n.pt&response-content-type=application%2Foctet-stream [following]
--2025-04-13 10:46:27-- https://objects.bgithub.xyz/github-production-release-asset-2e65be/521807533/34b70ade-b6eb-4179-a60f-d6494307226b?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=releaseassetproduction%2F20250413%2Fus-east-1%2Fs3%2Faws4_request&X-Amz-Date=20250413T024624Z&X-Amz-Expires=300&X-Amz-Signature=6ff6748bcd6a9ed033cd7a55a36d384b8d15a07da6ebb894f1b924da4b9aa56b5X-Amz-SignedHeaders=host&response-content-disposition=attachment%3B%20filename%3Dyolo11n.pt&response-content-type=application%2Foctet-stream
Resolving objects.bgithub.xyz (objects.bgithub.xyz)... 51.158.204.132
Connecting to objects.bgithub.xyz (objects.bgithub.xyz)|51.158.204.132|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 5613764 (5.4M) [application/octet-stream]
Saving to: 'yolo11n.pt'

yolo11n.pt          100%[=====>] 5.35M  2.26MB/s  in 2.4s

2025-04-13 10:46:32 (2.26 MB/s) - 'yolo11n.pt' saved [5613764/5613764]
```

建立 yolo11 的环境，python 设置为 3.10 版本

```
conda create -n yolo11 python=3.10 -y
```

```
(base) C:\Users\hchuang\ultralytics>conda create -n yolo11 python=3.10 -y
Collecting package metadata (current_repodata.json): done
Solving environment: done

==> WARNING: A newer version of conda exists. <==
  current version: 4.12.0
  latest version: 25.3.1

Please update conda by running

  $ conda update -n base -c defaults conda

## Package Plan ##

  environment location: C:\Users\hchuang\anaconda3\envs\yolo11

  added / updated specs:
    - python=3.10
```

激活该环境

```
conda activate yolo11
```

```
(base) C:\Users\hchuang\ultralytics>conda activate yolo11
(yolo11) C:\Users\hchuang\ultralytics>
```

安装依赖

计算机带独立显卡，并已经安装好 cuda 的，运行下面这条：

```
conda install pytorch torchvision torchaudio pytorch-cuda=12.4 -c pytorch -c nvidia -y
```

否则安装这条

```
conda install pytorch torchvision torchaudio
```

```
(yolo11) C:\Users\hchuang\ultralytics>conda install pytorch torchvision torchaudio pytorch-cuda=12.4 -c pytorch -c nvidia -y
Collecting package metadata (current_repodata.json): done
Solving environment: failed with initial frozen solve. Retrying with flexible solve.
Solving environment: failed with repodata from current_repodata.json, will retry with next repodata source.
Collecting package metadata (repodata.json): done
Solving environment: done

==> WARNING: A newer version of conda exists. <==
  current version: 4.12.0
  latest version: 25.3.1

Please update conda by running

  $ conda update -n base -c defaults conda

## Package Plan ##

  environment location: C:\Users\hchuang\anaconda3\envs\yolo11

  added / updated specs:
    - pytorch
    - pytorch-cuda=12.4
    - torchaudio
```

```
(print(torch.cuda.is_available()))
```

再安装

```
pip install onnx==1.18.0 onnxslim onnxruntime -i https://pypi.tuna.tsinghua.edu.cn/simple
```

```
pip install ultralytics -i https://pypi.tuna.tsinghua.edu.cn/simple
```

验证安装情况, 运行

```
yolo predict model=yololln.pt source=./ultralytics/assets/bus.jpg
```

```
(yolo11) C:\Users\hchuang\ultralytics>yolo predict model=yololln.pt source='sign.jpg'
Ultralytics 8.3.107 Python-3.10.16 torch-2.5.1 CUDA:0 (NVIDIA GeForce GTX 1070 with Max-Q Design, 8192MiB)
YOLO11n summary (fused): 100 layers, 2,616,248 parameters, 0 gradients, 6.5 GFLOPs

image 1/1 C:\Users\hchuang\ultralytics\sign.jpg: 384x640 4 persons, 3 motorcycles, 4 traffic lights, 87.3ms
Speed: 3.1ms preprocess, 87.3ms inference, 100.6ms postprocess per image at shape (1, 3, 384, 640)
Results saved to C:\Users\hchuang\ultralytics\runs\detect\predict
Learn more at https://docs.ultralytics.com/modes/predict
```

推理输出目录

在输出地方, 看推理结果

原图



推理结果



2) 训练模型

数据分割

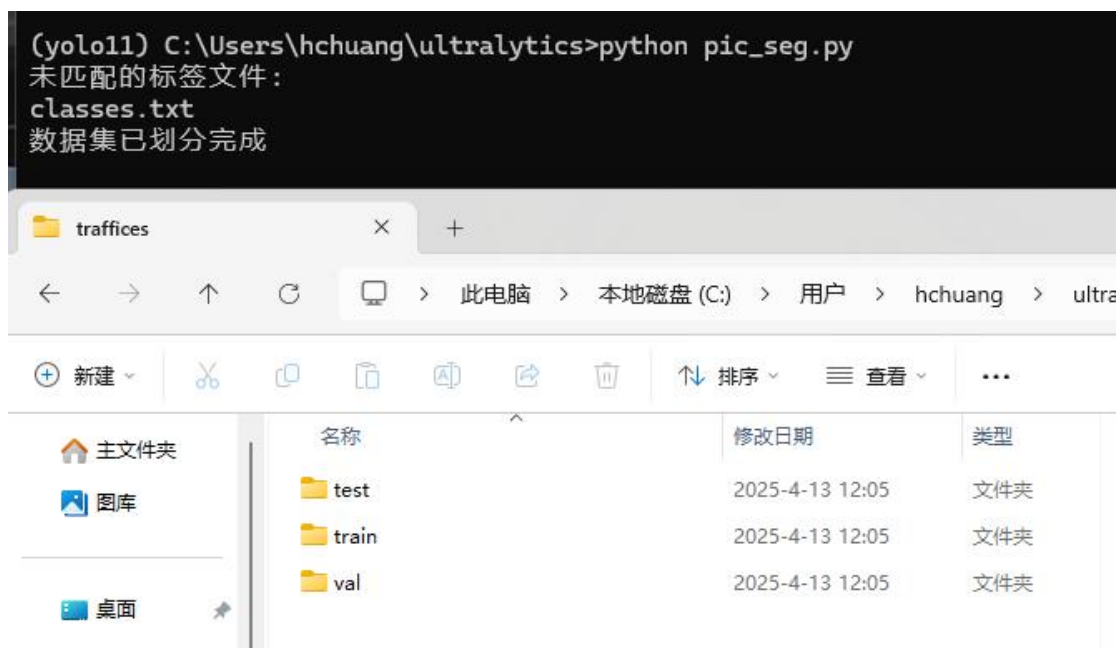
标注好的数据集按照一定比例分成训练集、验证集和测试集 (8:1:1), 把 pic_seg.py 拷贝到 ultralytics 目录下, 修改最后几行。

```
if __name__ == '__main__':
    file_path = r"c:\Users\hhch\VOC2007\JPEGImages" # 数据图片文件夹绝对路径
    label_path = r"c:\Users\hhch\VOC2007\Annotations" # 标签文件夹绝对路径
    new_file_path = r".\traffices" # 新数据存放位置
    split_data(file_path, label_path, new_file_path, train_rate=0.8, val_rate=0.1,
```

改成数据图片、标注文件夹的绝对路径。分割后的数据集放在 ultralytics 目录下。运行

```
python pic_seg.py
```

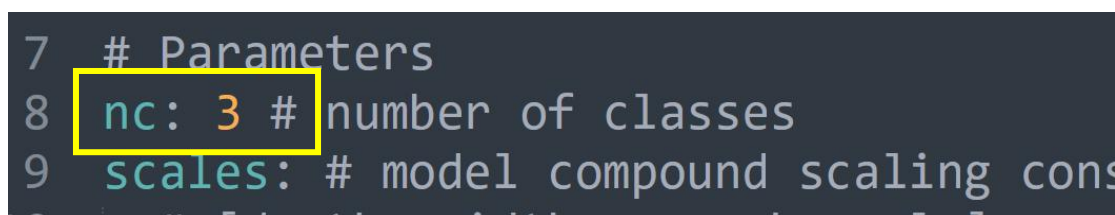
在 ultralytics 下, 生成 traffices 文件夹, 下面有三个子目录



拷贝 `ultralytics\ultralytics\cfg\models\11\` 下的 `yolo11.yaml` 到 `ultralytics` 目录下，并改名 `myyolo11n.yaml`



打开 `myyolo11n.yaml`，把 `nc:80`，改为 3，并存盘退出。



在 `ultralytics` 新建 `mydata.yaml`


```

1 train: ./traffices/train/images
2 val:   ./traffices/val/images
3 test:  ./traffices/test/images
4
5 # number of classes
6 nc: 3
7
8 # class names
9 names: ['green-light', 'blue-light', 'red-light']

```

执行（这是一条命令，如果是 cpu， device=cpu）：

```
yolo task=detect mode=train model=myyolol1n.yaml pretrained=yolol1n.pt
data=mydata.yaml epochs=300 imgsz=640 device=0 optimizer='SGD'
workers=8 batch=16 amp=True cache=False lr0=0.01
```

参数含义：

data='mydata.yaml'， #数据集配置文件的路径

epochs=300， #训练轮次总数

batch=16， #批量大小，即单次输入多少图片训练

imgsz=640， #训练图像尺寸

workers=8， #加载数据的工作线程数

device= 0， #指定训练的计算设备，无 nvidia 显卡则改为 'cpu'

optimizer='auto'， #训练使用优化器，可选 auto,SGD,Adam,AdamW 等

amp= True， #True 或者 False，解释为：自动混合精度(AMP) 训练

cache=False # True 在内存中缓存数据集图像，服务器推荐开启

```

(yolol11) C:\Users\hchuang\ultralytics>yolo task=detect mode=train model=myyolol1n.yaml pretrained=yolol1n.pt data=mydata
.yaml epochs=300 imgsz=640 device=0 optimizer='SGD' workers=8 batch=16 amp=True cache=False lr0=0.01
Transferred 448/499 items from pretrained weights
Ultralytics 8.3.237 Python-3.10.19 torch-2.5.1 CUDA:0 (NVIDIA GeForce GTX 1070 with Max-Q Design, 8192MiB)
engine\trainer: agnostic_nms=False, amp=True, augment=False, auto_augment=randaugument, batch=16, bgr=0.0, box=7.5, cache
=False, cfg=None, classes=None, close_mosaic=10, cls=0.5, compile=False, conf=None, copy_paste=0.0, copy_paste_mode=flip
, cos_lr=False, cutmix=0.0, data=mydata.yaml, degrees=0.0, deterministic=True, device=0, dfl=1.5, dnn=False, dropout=0.0
, dynamic=False, embed=None, epochs=300, erasing=0.4, exist_ok=False, flipplr=0.5, flipud=0.0, format=torchscript, fracti
on=1.0, freeze=None, half=False, hsv_h=0.015, hsv_s=0.7, hsv_v=0.4, imgsz=640, int8=False, iou=0.7, keras=False, kobj=1.
0, line_width=None, lr0=0.01, lrf=0.01, mask_ratio=4, max_det=300, mixup=0.0, mode=train, model=myyolol1n.yaml, momentum
=0.937, mosaic=1.0, multi_scale=False, name=train, nbs=64, nms=False, opset=None, optimize=False, optimizer=SGD, overlap
_mask=True, patience=100, perspective=0.0, plots=True, pose=12.0, pretrained=yolol1n.pt, profile=False, project=None, re
ct=False, resume=False, retina_masks=False, save=True, save_conf=False, save_crop=False, save_dir=C:\Users\hchuang\ultra
lytics\runs\detect\train, save_frames=False, save_json=False, save_period=-1, save_txt=False, scale=0.5, seed=0, shear=0
.0, show=False, show_boxes=True, show_conf=True, show_labels=True, simplify=True, single_cls=False, source=None, split=v
al, stream_buffer=False, task=detect, time=None, tracker=botssort.yaml, translate=0.1, val=True, verbose=True, vid_stride
=1, visualize=False, warmup_bias_lr=0.1, warmup_epochs=3.0, warmup_momentum=0.8, weight_decay=0.0005, workers=8, workspa
ce=None

from  n  params  module  arguments
0      -1  1      464    ultralytics.nn.modules.conv.Conv  [3, 16, 3, 2]
1      -1  1      4672   ultralytics.nn.modules.conv.Conv  [16, 32, 3, 2]
2      -1  1      6640   ultralytics.nn.modules.block.C3k2  [32, 64, 1, False, 0.25]
3      -1  1      36992  ultralytics.nn.modules.conv.Conv  [64, 64, 3, 2]

```

训练过程

Starting training for 300 epochs...

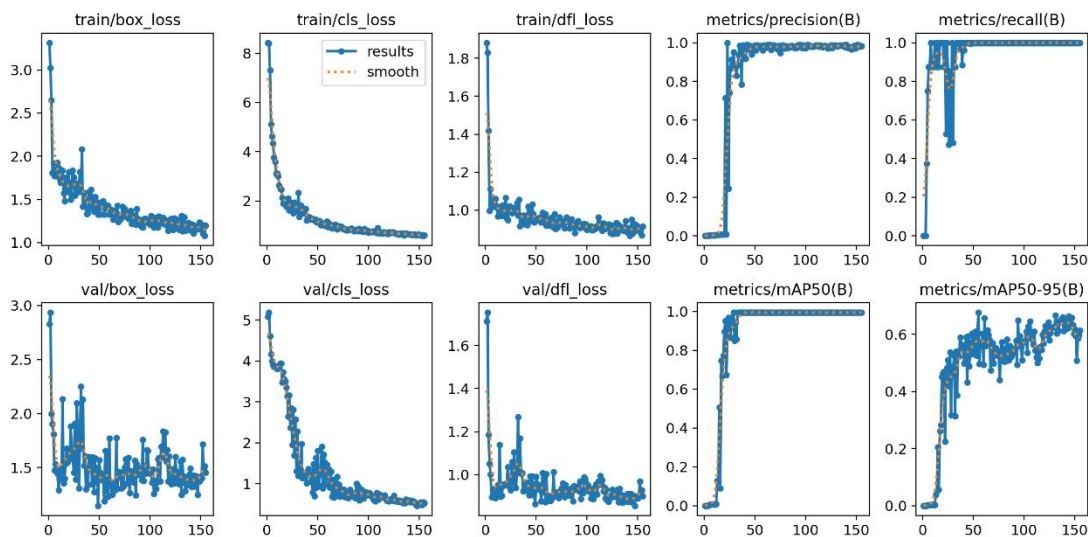
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
1/300	2.4G	3.03	9.007	1.717	31	640: 100%	4/4 1.4s/it 5.6s
	Class	Images	Instances	Box(P)	R	mAP50 mAP50-95): 100%	1/1 2.3it/s
	Class	Images	Instances	Box(P)	R	mAP50 mAP50-95): 100%	1/1 2.3it/s
0.4s	all	8	8	0	0	0	0
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
2/300	2.4G	2.871	7.929	1.873	27	640: 100%	4/4 2.3it/s 1.7s
	Class	Images	Instances	Box(P)	R	mAP50 mAP50-95): 100%	1/1 12.5it/s
	Class	Images	Instances	Box(P)	R	mAP50 mAP50-95): 100%	1/1 12.5it/s
0.1s	all	8	8	0	0	0	0
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
3/300	2.41G	2.517	7.538	1.433	26	640: 100%	4/4 2.3it/s 1.7s
	Class	Images	Instances	Box(P)	R	mAP50 mAP50-95): 100%	1/1 11.1it/s
	Class	Images	Instances	Box(P)	R	mAP50 mAP50-95): 100%	1/1 11.1it/s
0.1s	all	8	8	0	0	0	0
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
4/300	2.41G	2.02	5.851	1.172	23	640: 100%	4/4 2.3it/s 1.7s
	Class	Images	Instances	Box(P)	R	mAP50 mAP50-95): 100%	1/1 11.0it/s
	Class	Images	Instances	Box(P)	R	mAP50 mAP50-95): 100%	1/1 11.0it/s
0.1s	all	8	8	0.000876	0.125	0.000549	0.00022

不一定训练到 300 次，精度达到一定后，也会停止训练的

```
155 epochs completed in 0.097 hours.
Optimizer stripped from C:\Users\hchuang\ultralytics\runs\detect\train3\weights\last.pt, 5.5MB
Optimizer stripped from C:\Users\hchuang\ultralytics\runs\detect\train3\weights\best.pt, 5.5MB

Validating C:\Users\hchuang\ultralytics\runs\detect\train3\weights\best.pt...
Ultralytics 8.3.107 Python-3.10.16 torch-2.5.1 CUDA:0 (NVIDIA GeForce GTX 1070 with Max-Q Design, 8192MiB)
myYOLO11n summary (fused): 100 layers, 2,582,542 parameters, 0 gradients, 6.3 GFLOPs
Class      Images  Instances  Box(P)  R      mAP50  mAP50-95): 100% | 1/1 [00:00<0]
  all       8         8         0.976   1     0.995  0.677
green-light 4         4         0.988   1     0.995  0.72
red-light   4         4         0.964   1     0.995  0.633
Speed: 0.3ms preprocess, 3.9ms inference, 0.0ms loss, 2.2ms postprocess per image
Results saved to C:\Users\hchuang\ultralytics\runs\detect\train3
Learn more at https://docs.ultralytics.com/modes/train
```

训练结果所在目录



把训练结果，weights/best.pt 拷贝到 ultralytics 目录下。

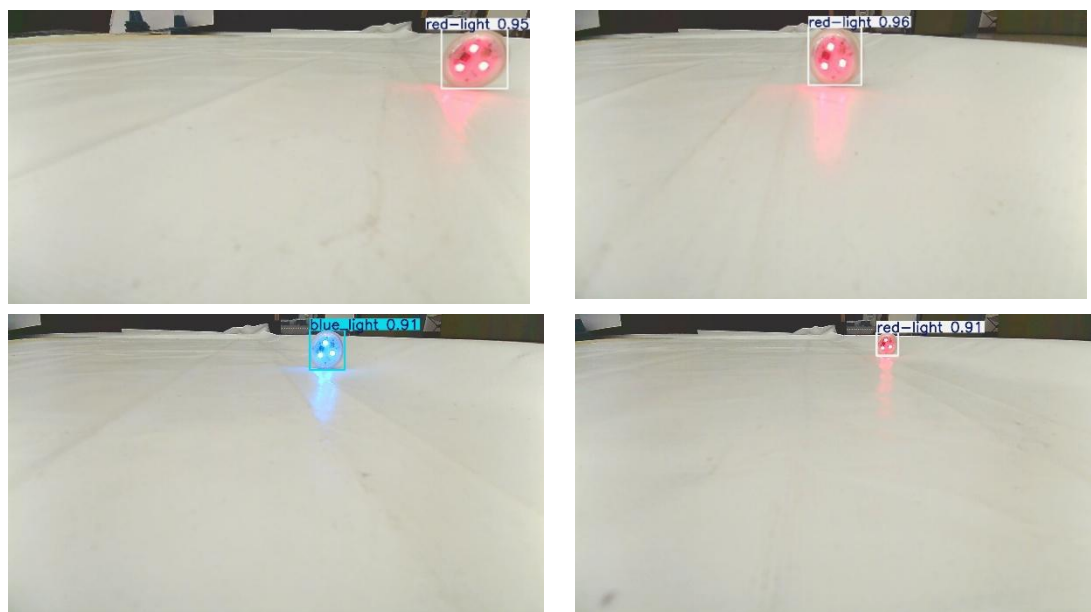
验证模型，执行

```
yolo predict model=best.pt source='./traffices/test/images'
```

观察推理结果是否正确

```
(yolo11) C:\Users\hchuang\ultralytics>yolo predict model=best.pt source='./traffics/test/images'
Ultralytics 8.3.107 Python-3.10.16 torch-2.5.1 CUDA:0 (NVIDIA GeForce GTX 1070 with Max-Q Design, 8192MiB)
myYOLO11n summary (fused): 100 layers, 2,582,542 parameters, 0 gradients, 6.3 GFLOPs

image 1/8 C:\Users\hchuang\ultralytics\traffics\test\images\2024-11-04-19_27_23_021170.jpg: 384x640 2 green-lights, 72.9ms
image 2/8 C:\Users\hchuang\ultralytics\traffics\test\images\2024-11-04-19_28_16_226749.jpg: 384x640 1 green-light, 16.5ms
image 3/8 C:\Users\hchuang\ultralytics\traffics\test\images\2024-11-04-19_29_27_134350.jpg: 384x640 1 red-light, 15.2ms
image 4/8 C:\Users\hchuang\ultralytics\traffics\test\images\2024-11-04-19_29_41_860913.jpg: 384x640 1 red-light, 15.8ms
image 5/8 C:\Users\hchuang\ultralytics\traffics\test\images\2024-11-04-19_30_08_802701.jpg: 384x640 1 red-light, 16.2ms
image 6/8 C:\Users\hchuang\ultralytics\traffics\test\images\2024-11-04-19_31_01_043866.jpg: 384x640 1 red-light, 16.6ms
image 7/8 C:\Users\hchuang\ultralytics\traffics\test\images\2024-11-04-19_31_02_110273.jpg: 384x640 1 red-light, 17.8ms
image 8/8 C:\Users\hchuang\ultralytics\traffics\test\images\2024-11-04-19_31_12_970692.jpg: 384x640 1 red-light, 17.4ms
Speed: 2.3ms preprocess, 23.5ms inference, 15.0ms postprocess per image at shape (1, 3, 384, 640)
Results saved to C:\Users\hchuang\ultralytics\runs\detect\predict3
💡 Learn more at https://docs.ultralytics.com/modes/predict
```



3) 模型转换成 onnx

把 export_monkey_patch.py 文件上传到 ultralytics 目录下;

运行

```
python export_monkey_patch.py --pt best.pt
```

```
(yolo11) C:\Users\hhch\ultralytics>python export_monkey_patch.py --pt best.pt
[Cauchy] Patched Attention: attn
[Cauchy] Patched Detect: 23
[Cauchy] Exporting model with opset=11, imgsz=[640, 640]...
Ultralytics 8.4.57 Python-3.10.20 torch-2.5.1 CPU (Intel Core(TM) i9-14900HX)
myYOLO11n summary (fused): 101 layers, 2,582,737 parameters, 0 gradients, 6.3 GFLOPs

PyTorch: starting from 'best.pt' with input shape (1, 3, 640, 640) BCHW and output shape(s) ((1, 80, 80, 3), (1, 80, 80, 64), (1, 40, 40, 3), (1, 40, 40, 64), (1, 20, 20, 3), (1, 20, 20, 64)) (5.2 MB)

ONNX: starting export with onnx 1.18.0 opset 11...
ONNX: export success 0.7s, saved as 'best.onnx' (9.9 MB)

Export complete (0.9s)
Results saved to C:\Users\hhch\ultralytics\best.onnx
Predict:      yolo predict task=detect model=best.onnx imgsz=640
Validate:     yolo val task=detect model=best.onnx imgsz=640 data=mydata.yaml
Visualize:    https://netron.app
```

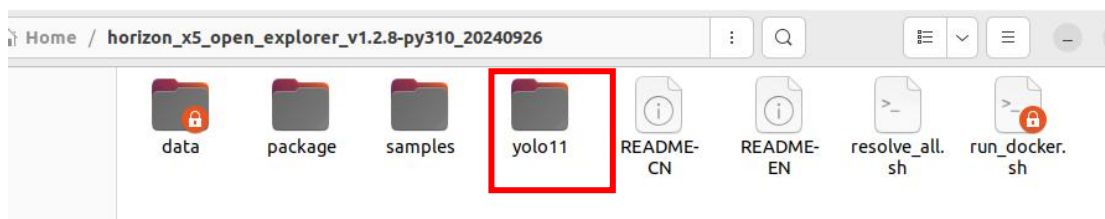
在 ultralytics 目录下, 生成 best.onnx 模型

4) 模型转成 bin

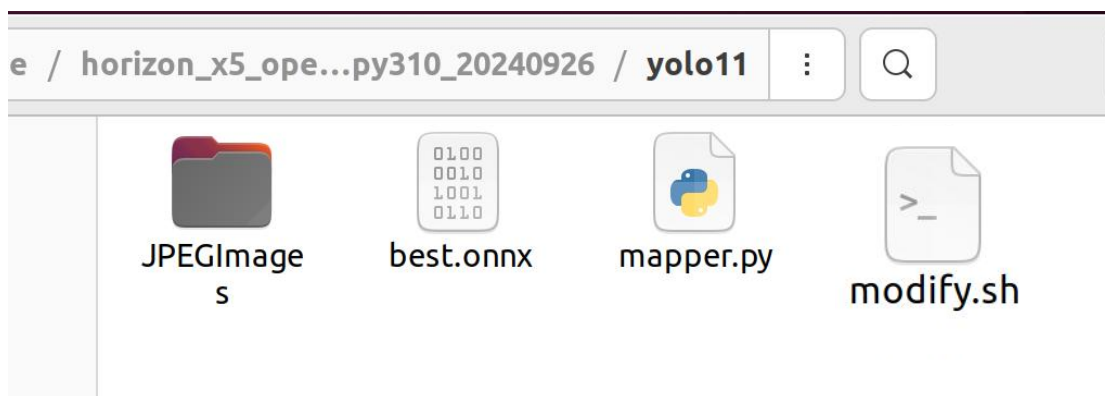
打开虚拟机 originbot

horizon_x5_open_explorer_v1.2.8-py310_20240926 目录下, 新建一个文件夹

yolo11



把 `best.onnx`、`mapper.py`、`modify.sh` 和标注图片的 `JPEGImages` 文件夹一起拷贝，黏贴到 `yolo11` 目录下



用热键 `ctr+alt+t` 开一个终端，输入

```
cd horizon_x5_open_explorer_v1.2.8-py310_20240926/
```

(用 `table` 键补齐)

```
sudo bash run_docker.sh /data cpu
```

```
originbot@originbot-virtual-machine:~$ cd horizon_x5_open_explorer_v1.2.8-py310_20240926/
originbot@originbot-virtual-machine:~/horizon_x5_open_explorer_v1.2.8-py310_20240926$ sudo bash
run_docker.sh /data cpu
[sudo] password for originbot:
root@da1221e76e7a:/open_explorer#
```

(密码: `ros`)

进入 `yolo11` 目录，运行

```
cd yolo11
```

```
python3 mapper.py --onnx best.onnx --cal-images ./JPEGImages/
```

再运行 `sh modify.sh`

生成 `best_bayese_640x640_nv12_modified.bin`，这是最终模型

对该模型进行验证，运行

```
hb_perf best_bayese_640x640_nv12_modified.bin
```

在

`hb_perf_result/best_bayese_640x640_nv12_modified/best_bayese_640x640_nv12_modified.png` 形状应该如下图所示

基于 YOLO11 识别红绿灯

